



Training Guide — Part 2

Chapters 9.5 · 9.6 · 10 · 11

Search Algorithms · Sort Algorithms · Multi-Threading · Concurrency

CONTINUATION OF THE 3-DAY INTENSIVE – ADVANCED CHAPTERS

9.5 Search Algorithms (Linear, Binary, Jump)

9.6 Sort Algorithms (Bubble, Quick, Merge, Heap)

10.0 Multi-Threading and Concurrency in C

11.0 Search and Sorting – Applied



Table of Contents

- [9.5 Search Algorithms](#)
 - [Linear Search — Brute Force](#)
 - [Binary Search — Divide and Conquer](#)
 - [Jump Search — Divide and Conquer](#)
- [9.6 Sort Algorithms](#)

- Bubble Sort — Exchanging
- Quick Sort — Divide and Conquer
- Merge Sort — Divide and Conquer
- Heap Sort — Selection
- 10.0 Multi-Threading and Concurrency in C
- 10.1 Basics of pthread Library
- 10.2 Thread Creation and Management
- 10.3 Synchronization Mechanisms in C
- 11.0 Search and Sorting Algorithms — Applied
- 11.1 Implementing Linear and Binary Search
- 11.2 Sorting Algorithms: Bubble, Quick, Merge Sort

9.5 Search Algorithms



Learning Objectives

- Understand what a search algorithm is and why the strategy matters
- Implement Linear, Binary, and Jump Search from scratch in C++
- Know which search to apply based on whether data is sorted
- Analyze and compare the time complexities of each approach

Introduction: The Search Problem

Every application searches for data constantly — finding a user in a database, checking if a word exists in a dictionary, locating a file on disk. The **search algorithm** is the strategy the computer uses to find what it's looking for.

The fundamental question is: **how do you minimize the number of comparisons?**

THE SEARCH PROBLEM:

Given: An array of elements and a target value.

Find: The INDEX of the target (or report "not found").

Example: Array = [15, 3, 42, 8, 27, 64, 11, 99]

Target = 27

Answer: index 4

STRATEGIES:

Brute Force → Check every element. Always works.

Divide & Conquer → Split in half each time. FAST.

Search Algorithm Comparison

Algorithm	Time (Best)	Time (Worst)	Prerequisite
Linear Search	$O(1)$	$O(n)$	None
Binary Search	$O(1)$	$O(\log n)$	SORTED array
Jump Search	$O(\sqrt{n})$	$O(\sqrt{n})$	SORTED array

For $n = 1,000,000$ elements:

Linear → up to 1,000,000 comparisons

Jump → up to ~1,000 comparisons

Binary → up to ~20 comparisons ← incredible!

Linear Search — Brute Force Strategy

Concept

Linear search is the simplest possible strategy: **start at the beginning and check every element one by one** until the target is found or the array is exhausted.

ARRAY: [5, 12, 7, 23, 45, 8, 30]

TARGET: 23

Step 1: Check index 0 → 5 ≠ 23 → continue

Step 2: Check index 1 → 12 ≠ 23 → continue

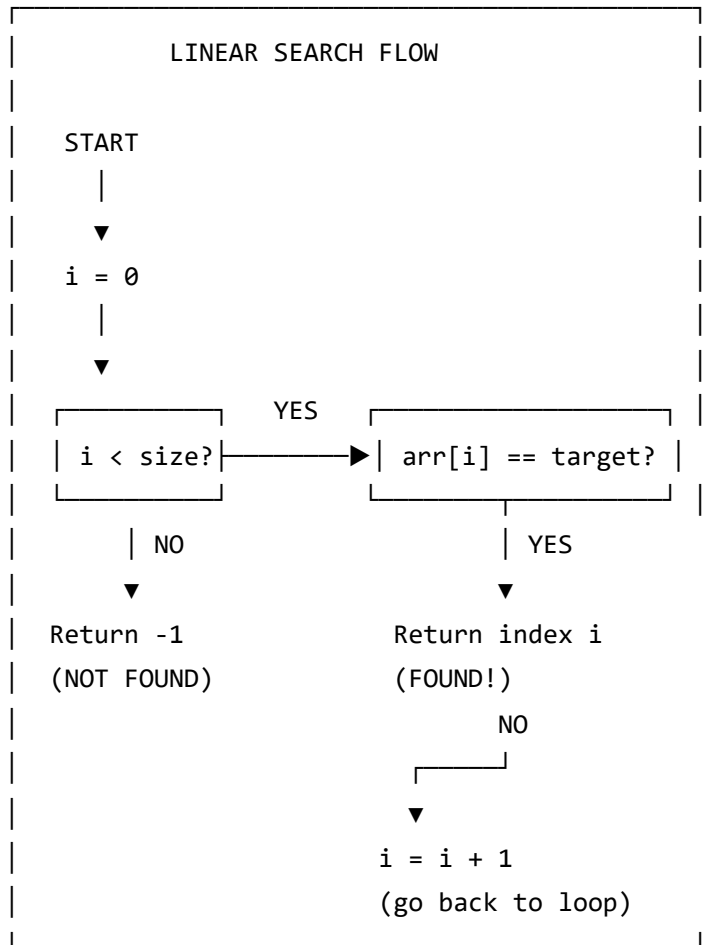
Step 3: Check index 2 → 7 ≠ 23 → continue

Step 4: Check index 3 → 23 = 23 → FOUND at index 3! ✓

Worst case (not found): checks ALL 7 elements.

Best case (first element): checks 1 element.

Flow Chart



Implementation

```
#include <stdio.h>
```

```
// — LINEAR SEARCH —————  
// Strategy: Brute Force – check each element sequentially  
// Works on: Sorted AND unsorted arrays  
// Time:  $O(n)$  worst case,  $O(1)$  best case  
// Space:  $O(1)$  – no extra memory needed  
//  
// Returns: index of target if found, -1 if not found  
int linearSearch(int arr[], int size, int target) {  
    for (int i = 0; i < size; i++) {  
        if (arr[i] == target) {  
            return i;    // Target found at index i  
        }  
    }  
    return -1;    // Target not in array  
}
```

```
// — FIND ALL OCCURRENCES —————  
// Enhanced version: prints ALL positions where target appears  
void linearSearchAll(int arr[], int size, int target) {  
    int found = 0;  
    printf("Searching for %d...\n", target);  
  
    for (int i = 0; i < size; i++) {  
        if (arr[i] == target) {  
            printf(" Found at index: %d\n", i);  
            found++;  
        }  
    }  
}
```

```
    }

    if (found == 0)
        printf(" Not found.\n");
    else
        printf(" Total occurrences: %d\n", found);
}

// — SENTINEL LINEAR SEARCH —————
// Optimization: place target at end as sentinel to remove bounds check
// Saves one comparison per iteration – slightly faster in practice
int sentinelSearch(int arr[], int size, int target) {
    int last = arr[size - 1];    // Save last element
    arr[size - 1] = target;      // Place sentinel at end

    int i = 0;
    while (arr[i] != target)      // No need to check i < size!
        i++;

    arr[size - 1] = last;        // Restore last element

    // Check if found genuinely or just hit the sentinel
    if (i < size - 1 || arr[size - 1] == target)
        return i;
    return -1;
}

int main() {
    int data[] = {64, 25, 12, 22, 11, 90, 5, 42, 33};
    int n = sizeof(data) / sizeof(data[0]);

    printf("=== Linear Search Demo ===\n\n");
```

```
printf("Array: ");
for (int i = 0; i < n; i++) printf("%d ", data[i]);
printf("\n\n");

// Search 1: found
int idx = linearSearch(data, n, 22);
printf("Search for 22: %s\n",
       idx != -1 ? "Found" : "Not Found");
if (idx != -1) printf(" → Index: %d\n", idx);

// Search 2: not found
idx = linearSearch(data, n, 100);
printf("Search for 100: %s\n\n",
       idx != -1 ? "Found" : "Not Found");

// Find all occurrences
int data2[] = {3, 7, 3, 9, 3, 1, 3};
int n2 = 7;
printf("Array with duplicates: ");
for (int i = 0; i < n2; i++) printf("%d ", data2[i]);
printf("\n");
linearSearchAll(data2, n2, 3);

return 0;
}
```

Output:

=== Linear Search Demo ===

Array: 64 25 12 22 11 90 5 42 33

Search for 22: Found

→ Index: 3

Search for 100: Not Found

Array with duplicates: 3 7 3 9 3 1 3

Searching for 3...

Found at index: 0

Found at index: 2

Found at index: 4

Found at index: 6

Total occurrences: 4

Real-World Use Cases

- ✓ Searching an unsorted list of contacts by name
- ✓ Finding a specific log entry by error code
- ✓ Checking if a value exists in a small dataset ($n < 100$)
- ✓ Searching a linked list (no random access → must go element by element)

Binary Search — Divide and Conquer Strategy

Concept

Binary search is one of the most elegant algorithms in computer science. It requires a **sorted array** and works by **halving the search space** at every step.

Think of finding a word in a dictionary: you don't start from "A" and flip one page at a time. You open to the middle, see if your word comes before or after, then search the relevant half. Binary search does exactly this.

SORTED ARRAY: [2, 5, 8, 12, 16, 23, 38, 45, 56, 72, 91]

TARGET: 23

Indices: 0 1 2 3 4 5 6 7 8 9 10

— Iteration 1 —

low=0, high=10, mid=5

arr[5] = 23 = target → FOUND at index 5! ✓

— If target was 38 —

Iter 1: low=0, high=10, mid=5 → arr[5]=23 < 38 → search RIGHT half

Iter 2: low=6, high=10, mid=8 → arr[8]=56 > 38 → search LEFT half

Iter 3: low=6, high=7, mid=6 → arr[6]=38 = target → FOUND! ✓

Each iteration ELIMINATES HALF the remaining elements!

The Power of Divide and Conquer

n = 1,024 elements:

After 1 step: 512 remain

After 2 steps: 256 remain

After 3 steps: 128 remain

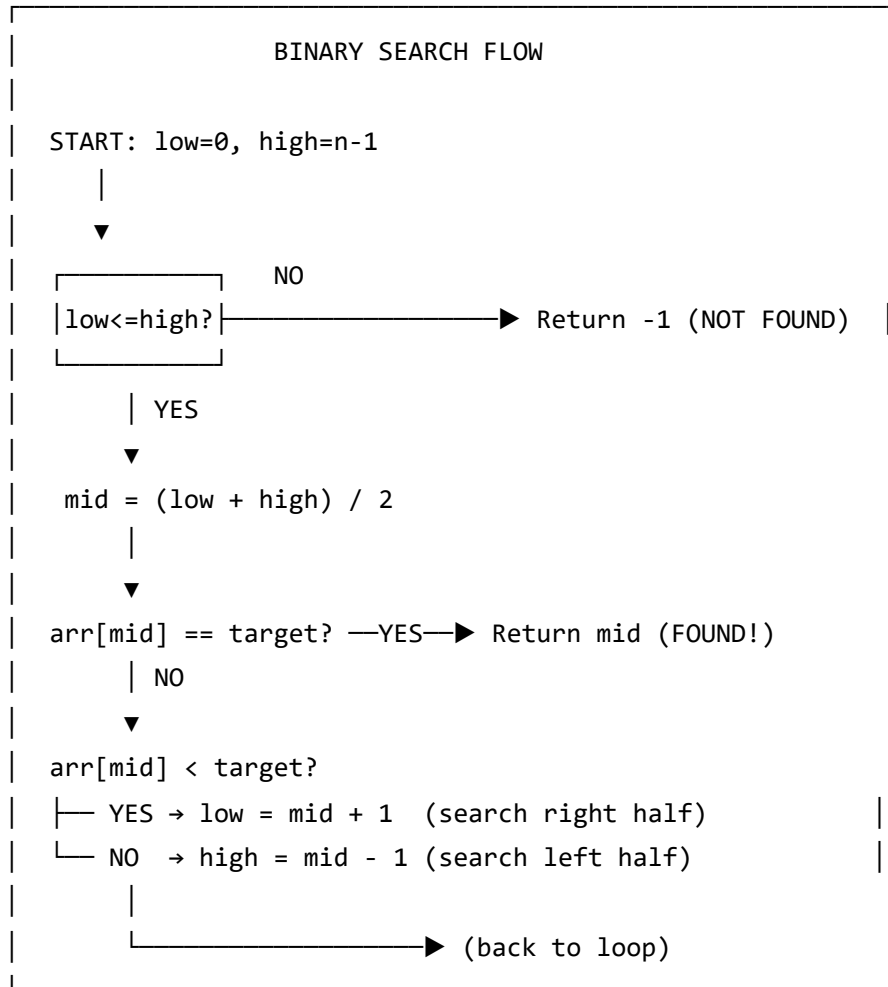
...

After 10 steps: 1 remains ← DONE!

$\log_2(1024) = 10$. Binary search needs at most 10 steps!

Linear search needs up to 1,024 steps.

Flow Chart



Implementation — Iterative and Recursive

```
#include <stdio.h>

// ——— BINARY SEARCH (ITERATIVE) ———
// Strategy: Divide and Conquer
// Requires: SORTED array (ascending order)
// Time:  $O(\log n)$  – halves the search space each iteration
// Space:  $O(1)$  – no recursion stack needed
//
// Returns: index of target if found, -1 if not found
int binarySearchIterative(int arr[], int size, int target) {
    int low  = 0;           // Left boundary of search window
    int high = size - 1;    // Right boundary of search window

    while (low <= high) {
        // IMPORTANT: Use (low + high)/2 carefully to avoid integer overflow
        // Safe formula: mid = low + (high - low) / 2
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;      // ✓ Found!
        } else if (arr[mid] < target) {
            low = mid + 1;    // Target is in RIGHT half
        } else {
            high = mid - 1;   // Target is in LEFT half
        }
    }
}
```

```
    return -1;    // Not found
}

// — BINARY SEARCH (RECURSIVE) —————
// Same logic but expressed recursively – elegant and intuitive
// Time: O(log n)
// Space: O(log n) – recursion stack depth
int binarySearchRecursive(int arr[], int low, int high, int target) {
    // Base case: search window is empty
    if (low > high) return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == target)
        return mid;                // Found!

    if (arr[mid] < target)
        return binarySearchRecursive(arr, mid + 1, high, target); // Search right

    return binarySearchRecursive(arr, low, mid - 1, target);    // Search left
}

// — FIND FIRST OCCURRENCE (for duplicate elements) —————
// Standard binary search finds ANY occurrence of duplicates.
// This version finds the FIRST (leftmost) occurrence.
int findFirst(int arr[], int size, int target) {
    int low = 0, high = size - 1, result = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
```

```
        result = mid;           // Record this position
        high = mid - 1;         // But keep searching LEFT for earlier one
    } else if (arr[mid] < target) {
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}
return result;
}
```

```
// — STEP-BY-STEP TRACE —————
```

```
// Educational version: shows every step of the algorithm
```

```
int binarySearchVerbose(int arr[], int size, int target) {
    int low = 0, high = size - 1, step = 1;

    printf("Searching for %d in array of %d elements:\n", target, size);

    while (low <= high) {
        int mid = low + (high - low) / 2;
        printf(" Step %d: low=%d, high=%d, mid=%d → arr[%d]=%d ",
            step++, low, high, mid, mid, arr[mid]);

        if (arr[mid] == target) {
            printf("→ MATCH! Found at index %d\n", mid);
            return mid;
        } else if (arr[mid] < target) {
            printf("→ Too small, search RIGHT\n");
            low = mid + 1;
        } else {
            printf("→ Too large, search LEFT\n");
            high = mid - 1;
        }
    }
}
```

```
    }  
}  
printf("  Target not found.\n");  
return -1;  
}  
  
int main() {  
    int sorted[] = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72, 91};  
    int n = sizeof(sorted) / sizeof(sorted[0]);  
  
    printf("=== Binary Search Demo ===\n\n");  
    printf("Sorted Array: ");  
    for (int i = 0; i < n; i++) printf("%d ", sorted[i]);  
    printf("\n\n");  
  
    // Iterative search  
    printf("--- Iterative Binary Search ---\n");  
    int idx = binarySearchIterative(sorted, n, 38);  
    printf("Search for 38: %s (index %d)\n\n",  
        idx != -1 ? "FOUND" : "NOT FOUND", idx);  
  
    // Recursive search  
    printf("--- Recursive Binary Search ---\n");  
    idx = binarySearchRecursive(sorted, 0, n - 1, 5);  
    printf("Search for 5: %s (index %d)\n\n",  
        idx != -1 ? "FOUND" : "NOT FOUND", idx);  
  
    // Verbose trace  
    printf("--- Step-by-step trace ---\n");  
    binarySearchVerbose(sorted, n, 56);  
    printf("\n");  
    binarySearchVerbose(sorted, n, 100);  
}
```

```
// First occurrence
int withDups[] = {1, 2, 3, 3, 3, 4, 5};
printf("\n--- First Occurrence (duplicates) ---\n");
printf("Array: 1 2 3 3 3 4 5\n");
printf("First '3' at index: %d\n", findFirst(withDups, 7, 3));

return 0;
}
```

Output:

=== Binary Search Demo ===

Sorted Array: 2 5 8 12 16 23 38 45 56 72 91

--- Iterative Binary Search ---

Search for 38: FOUND (index 6)

--- Recursive Binary Search ---

Search for 5: FOUND (index 1)

--- Step-by-step trace ---

Searching for 56 in array of 11 elements:

Step 1: low=0, high=10, mid=5 → arr[5]=23 → Too small, search RIGHT

Step 2: low=6, high=10, mid=8 → arr[8]=56 → MATCH! Found at index 8

Searching for 100 in array of 11 elements:

Step 1: low=0, high=10, mid=5 → arr[5]=23 → Too small, search RIGHT

Step 2: low=6, high=10, mid=8 → arr[8]=56 → Too small, search RIGHT

Step 3: low=9, high=10, mid=9 → arr[9]=72 → Too small, search RIGHT

Step 4: low=10, high=10, mid=10 → arr[10]=91 → Too small, search RIGHT

Target not found.

--- First Occurrence (duplicates) ---

Array: 1 2 3 3 3 4 5

First '3' at index: 2

Jump Search — Divide and Conquer Strategy

Concept

Jump Search is a middle ground between Linear Search ($O(n)$) and Binary Search ($O(\log n)$). It works on sorted arrays by **jumping ahead by fixed blocks** of size $\sqrt{n}$, then doing a linear search within the relevant block.

SORTED ARRAY (n=16, block size= $\sqrt{16}=4$):

Index: 0 1 2 3 | 4 5 6 7 | 8 9 10 11 | 12 13 14 15

Values: [1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31]

TARGET: 19

Jump 1: Check arr[0] = 1 < 19 → jump forward

Jump 2: Check arr[4] = 9 < 19 → jump forward

Jump 3: Check arr[8] = 17 < 19 → jump forward

Jump 4: Check arr[12] = 25 > 19 → STOP! Target is in PREVIOUS block

Linear search backward from index 12 to 8:

arr[11] = 23 ≠ 19

arr[10] = 21 ≠ 19

arr[9] = 19 = TARGET → FOUND at index 9! ✓

Choosing the Block Size

WHY BLOCK SIZE = $\sqrt{n}$?

Total comparisons = jumps + linear_search
= $(n/k) + k$

where k = block size

To minimize: $d/dk [(n/k) + k] = 0$

$$-n/k^2 + 1 = 0$$

$$k = \sqrt{n} \leftarrow \text{OPTIMAL block size!}$$

For $n=100$: $k=10$. At most $10+10=20$ comparisons.

Linear would need up to 100.

Implementation

```
#include <stdio.h>
#include <math.h>    // For sqrt()

// — JUMP SEARCH —————
// Strategy: Divide and Conquer (block jumping + linear scan)
// Requires: SORTED array (ascending)
// Time:  $O(\sqrt{n})$  – better than  $O(n)$ , worse than  $O(\log n)$ 
// Space:  $O(1)$ 
// Best for: Large sorted arrays where binary search cache misses matter
//
// Returns: index of target if found, -1 if not found
int jumpSearch(int arr[], int size, int target) {
    // Step 1: Calculate optimal block size
    int step = (int)sqrt((double)size);    //  $\sqrt{n}$ 
    int prev = 0;                          // Start of current block

    // Step 2: Jump forward until we overshoot or reach end
    printf("  Block size ( $\sqrt{n} \approx$  %d)\n", size, step);

    while (arr[(step < size ? step : size) - 1] < target) {
        printf("  Jump: arr[%d]=%d < %d → jump forward\n",
            (step < size ? step : size) - 1,
            arr[(step < size ? step : size) - 1],
            target);
        prev = step;
        step += (int)sqrt((double)size);

        if (prev >= size) return -1;    // Jumped past the end
    }
}
```

```
printf(" Overshot at index %d → linear search from %d\n",
      (step < size ? step : size) - 1, prev);

// Step 3: Linear search within the identified block
while (arr[prev] < target) {
    printf(" Linear: arr[%d]=%d < %d → step forward\n",
          prev, arr[prev], target);
    prev++;
    if (prev == (step < size ? step : size)) return -1;
}

// Step 4: Check if we found the target
if (arr[prev] == target) {
    printf(" Found at index %d!\n", prev);
    return prev;
}

return -1;
}

int main() {
    int sorted[] = {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610};
    int n = sizeof(sorted) / sizeof(sorted[0]);

    printf("=== Jump Search Demo ===\n\n");
    printf("Array (%d elements): ", n);
    for (int i = 0; i < n; i++) printf("%d ", sorted[i]);
    printf("\n\n");

    printf("--- Searching for 55 ---\n");
    int result = jumpSearch(sorted, n, 55);
```

```

printf("Result: %s\n\n", result != -1 ? "FOUND" : "NOT FOUND");

printf("--- Searching for 100 ---\n");
result = jumpSearch(sorted, n, 100);
printf("Result: %s\n", result != -1 ? "FOUND" : "NOT FOUND");

return 0;
}

```

Search Algorithm Comparison — Final Summary

WHEN TO USE WHICH SEARCH	
Condition	Best Choice
Unsorted data	Linear Search (only option)
Sorted, $n < 50$	Linear Search (overhead not worth it)
Sorted, $n > 50$	Binary Search (fastest: $O(\log n)$)
Sorted, disk I/O	Jump Search (sequential access friendlier)
Need ALL hits	Linear Search (binary finds one, not all)
Linked List	Linear Search (no random access available)



Common Mistakes in Search

- ✗ Using Binary Search on UNSORTED data → WRONG results, no crash
- ✗ Integer overflow: $\text{mid} = (\text{low} + \text{high}) / 2$
Fix: $\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$
- ✗ Off-by-one: `while(low < high)` misses the last element
Fix: always use `while(low <= high)`
- ✗ Infinite loop in recursive binary search (no base case)
Fix: always check if `(low > high)` return -1;



Section 9.5 — Exercises

Exercise 1 (Beginner): Implement linear search on a linked list. Why can't you use binary search here?

Exercise 2 (Intermediate): Given a sorted array `{1,3,5,7,9,11,13,15}`, trace through binary search manually (on paper) to find 11. How many comparisons does it take?

Exercise 3 (Advanced): Implement a function `countOccurrences(arr, n, target)` using binary search. Find the first and last occurrence, then compute `last - first + 1`.

9.6 Sort Algorithms

Learning Objectives

- Understand why sorting matters and the role of problem strategy
- Implement Bubble Sort, Quick Sort, Merge Sort, and Heap Sort
- Trace through each algorithm step by step with ASCII diagrams
- Compare algorithms by time complexity, space use, and stability

Introduction: Why Sort?

Sorting is one of the most fundamental operations in computing. A sorted array enables binary search, makes duplicates visible, and is a prerequisite for many other algorithms.

Sorting algorithms are classified by STRATEGY:

EXCHANGING → Compare and swap adjacent elements

Example: Bubble Sort

DIVIDE & CONQUER → Recursively split and merge/pivot

Examples: Quick Sort, Merge Sort

SELECTION → Build a structure (heap), extract max/min

Example: Heap Sort

Sort Algorithm Comparison

Algorithm	Best Time	Avg Time	Worst Time	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Stable = equal elements maintain their original relative order.

💡 **What does Stable mean?** If you sort [(Alice,3), (Bob,3), (Carol,1)] by score, a stable sort gives [(Carol,1), (Alice,3), (Bob,3)] — Alice still comes before Bob (as in the original). Unstable may give Bob before Alice.

Bubble Sort — Exchanging Strategy

Concept

Bubble Sort repeatedly **compares adjacent pairs** and swaps them if they are in the wrong order. After each pass, the largest unsorted element "bubbles up" to its correct position at the end.

UNSORTED: [64, 34, 25, 12, 22, 11, 90]

— PASS 1 —

[64, 34, 25, 12, 22, 11, 90]

64>34 → swap → [34, 64, 25, 12, 22, 11, 90]

64>25 → swap → [34, 25, 64, 12, 22, 11, 90]

64>12 → swap → [34, 25, 12, 64, 22, 11, 90]

64>22 → swap → [34, 25, 12, 22, 64, 11, 90]

64>11 → swap → [34, 25, 12, 22, 11, 64, 90]

64<90 → keep → [34, 25, 12, 22, 11, 64, 90]

↑ 90 is in final position

— PASS 2 —

...continues until fully sorted.

After each pass, the next largest element settles into place.

Implementation

```
#include <stdio.h>

// Helper to print array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

// Helper to swap two integers
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// — BUBBLE SORT —————
// Strategy: Exchanging – compare and swap adjacent pairs
// Time:  $O(n^2)$  average/worst,  $O(n)$  best (already sorted)
// Space:  $O(1)$  – in-place sorting
// Stable: YES
void bubbleSort(int arr[], int n) {
    for (int pass = 0; pass < n - 1; pass++) {

        // Optimization: track if any swap happened this pass
        int swapped = 0;

        // After 'pass' iterations, the last 'pass' elements are sorted
        for (int i = 0; i < n - 1 - pass; i++) {
            if (arr[i] > arr[i + 1]) {
```

```
        swap(&arr[i], &arr[i + 1]);
        swapped = 1;
    }
}

// If no swaps occurred, array is already sorted – EARLY EXIT!
if (!swapped) {
    printf(" Early exit after pass %d (array sorted!)\n", pass + 1);
    break;
}

printf(" After pass %d: ", pass + 1);
printArray(arr, n);
}
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("=== Bubble Sort Demo ===\n\n");
    printf("Before: "); printArray(arr, n);
    printf("\nSorting...\n");
    bubbleSort(arr, n);
    printf("\nAfter: "); printArray(arr, n);

    // Demo early exit on already-sorted array
    printf("\n--- Early Exit Demo ---\n");
    int sorted[] = {1, 2, 3, 4, 5};
    printf("Before: "); printArray(sorted, 5);
    bubbleSort(sorted, 5);
    printf("After: "); printArray(sorted, 5);
```

```
    return 0;  
}
```

Output:

=== Bubble Sort Demo ===

Before: 64 34 25 12 22 11 90

Sorting...

After pass 1: 34 25 12 22 11 64 90

After pass 2: 25 12 22 11 34 64 90

After pass 3: 12 22 11 25 34 64 90

After pass 4: 12 11 22 25 34 64 90

After pass 5: 11 12 22 25 34 64 90

After: 11 12 22 25 34 64 90

--- Early Exit Demo ---

Before: 1 2 3 4 5

Early exit after pass 1 (array sorted!)

After: 1 2 3 4 5

Quick Sort — Divide and Conquer Strategy

Concept

Quick Sort is one of the fastest sorting algorithms in practice. It works by:

1. Choosing a **pivot** element
2. **Partitioning** the array so everything less than pivot is to its left, and everything greater is to its right
3. **Recursively** sorting the left and right sub-arrays

ARRAY: [10, 80, 30, 90, 40, 50, 70]

PIVOT: 70 (last element)

— PARTITION STEP —

Pointer *i* starts before the array.

For each element *j*:

if $\text{arr}[j] \leq \text{pivot} \rightarrow i++, \text{swap arr}[i] \text{ and } \text{arr}[j]$

$j=0$: $\text{arr}[0]=10 \leq 70 \rightarrow i=0, \text{swap}(\text{arr}[0], \text{arr}[0]) \rightarrow [10, 80, 30, 90, 40, 50, 70]$

$j=1$: $\text{arr}[1]=80 > 70 \rightarrow \text{no action} \rightarrow [10, 80, 30, 90, 40, 50, 70]$

$j=2$: $\text{arr}[2]=30 \leq 70 \rightarrow i=1, \text{swap}(\text{arr}[1], \text{arr}[2]) \rightarrow [10, 30, 80, 90, 40, 50, 70]$

$j=3$: $\text{arr}[3]=90 > 70 \rightarrow \text{no action}$

$j=4$: $\text{arr}[4]=40 \leq 70 \rightarrow i=2, \text{swap}(\text{arr}[2], \text{arr}[4]) \rightarrow [10, 30, 40, 90, 80, 50, 70]$

$j=5$: $\text{arr}[5]=50 \leq 70 \rightarrow i=3, \text{swap}(\text{arr}[3], \text{arr}[5]) \rightarrow [10, 30, 40, 50, 80, 90, 70]$

Place pivot: $\text{swap}(\text{arr}[i+1], \text{pivot}) \rightarrow [10, 30, 40, 50, 70, 90, 80]$

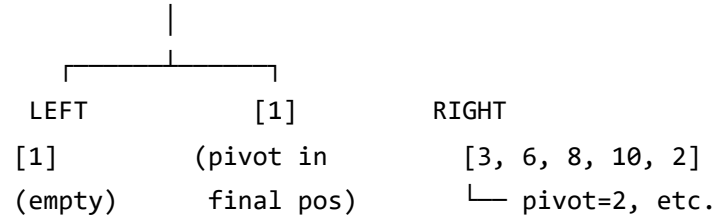
↑ PIVOT IS NOW IN FINAL POSITION

Recursively sort LEFT [10, 30, 40, 50] and RIGHT [90, 80]

Divide and Conquer Tree

QUICK SORT RECURSION TREE:

[3, 6, 8, 10, 1, 2, 1] (pivot=1, last element)



Eventually all sub-arrays have 0 or 1 element → sorted!

Implementation

```
#include <stdio.h>

void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

// — PARTITION —————
// Places pivot in its correct sorted position.
// Everything left of pivot < pivot, everything right > pivot.
// Returns: the final index of the pivot
int partition(int arr[], int low, int high) {
    int pivot = arr[high];    // Choose last element as pivot
    int i = low - 1;          // i tracks where to place next "small" element

    printf("  Partitioning [%d..%d], pivot=%d\n", low, high, pivot);

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    // Place pivot in its correct position
    swap(&arr[i + 1], &arr[high]);

    printf("  After partition: ");
```

```

    printArray(arr, high + 2 > 10 ? 10 : high + 2);    // show subset

    return i + 1;    // Return pivot's final index
}

// — QUICK SORT —————
// Strategy: Divide and Conquer – partition around pivot, recurse
// Time: O(n log n) average, O(n2) worst (sorted input with bad pivot)
// Space: O(log n) recursion stack
// Stable: NO (swaps can change equal elements' order)
//
// Tip: Use randomized pivot or median-of-three to avoid worst case
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // Partition and get pivot's final position
        int pivotIndex = partition(arr, low, high);

        // Recursively sort left and right of pivot
        quickSort(arr, low, pivotIndex - 1);    // Left sub-array
        quickSort(arr, pivotIndex + 1, high);    // Right sub-array
    }
}

// — RANDOMIZED QUICK SORT (BETTER PIVOT) —————
// Avoids worst-case O(n2) on sorted/reverse-sorted inputs
#include <stdlib.h>
#include <time.h>

int partitionRandom(int arr[], int low, int high) {
    // Randomly pick a pivot and swap to end
    int randIdx = low + rand() % (high - low + 1);
    swap(&arr[randIdx], &arr[high]);

```

```
        return partition(arr, low, high);
    }

void quickSortRandom(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partitionRandom(arr, low, high);
        quickSortRandom(arr, low, pivotIndex - 1);
        quickSortRandom(arr, pivotIndex + 1, high);
    }
}

int main() {
    int arr[] = {10, 80, 30, 90, 40, 50, 70};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("=== Quick Sort Demo ===\n\n");
    printf("Before: "); printArray(arr, n);
    printf("\nPartition trace:\n");
    quickSort(arr, 0, n - 1);
    printf("\nAfter: "); printArray(arr, n);

    return 0;
}
```

Merge Sort — Divide and Conquer Strategy

Concept

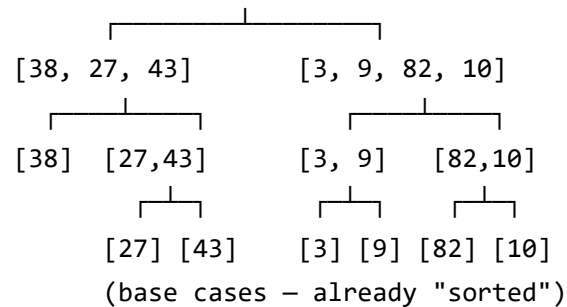
Merge Sort follows a clean two-phase approach:

1. **Divide:** Recursively split the array in half until each piece has one element (trivially sorted)
2. **Merge:** Combine pairs of sorted pieces into larger sorted pieces

ARRAY: [38, 27, 43, 3, 9, 82, 10]

— DIVIDE PHASE —

[38, 27, 43, 3, 9, 82, 10]



— MERGE PHASE —

Merge [27][43] → [27, 43]

Merge [38][27,43] → [27, 38, 43]

Merge [3][9] → [3, 9]

Merge [82][10] → [10, 82]

Merge [3,9][10,82] → [3, 9, 10, 82]

Merge [27,38,43][3,9,10,82] → [3, 9, 10, 27, 38, 43, 82] ✓

The Merge Operation

Merging two SORTED halves: [3, 27, 38] and [9, 10, 43, 82]

Left ptr (l): 3 → compare → smaller, take from left → [3, ...]

Left ptr (l): 27 → compare → 27 vs 9: 9 smaller, take → [..., 9, ...]

Left ptr (l): 27 → compare → 27 vs 10: 10 smaller → [..., 10, ...]

Left ptr (l): 27 → compare → 27 vs 43: 27 smaller → [..., 27, ...]

...and so on until both halves are exhausted.

Result: [3, 9, 10, 27, 38, 43, 82] ✓

Implementation

```
#include <stdio.h>
#include <stdlib.h>

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

// ——— MERGE —————
// Merges two sorted sub-arrays: arr[left..mid] and arr[mid+1..right]
// Uses a temporary array to hold merged result, then copies back
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;        // Size of left sub-array
    int n2 = right - mid;           // Size of right sub-array

    // Allocate temporary arrays
    int* L = (int*)malloc(n1 * sizeof(int));
    int* R = (int*)malloc(n2 * sizeof(int));

    // Copy data into temp arrays
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    printf("  Merging [");
    for (int i = 0; i < n1; i++) printf("%d%s", L[i], i<n1-1?" ":"");
    printf("] + [");
    for (int j = 0; j < n2; j++) printf("%d%s", R[j], j<n2-1?" ":"");
    printf("] → ");
}
```

```
// Merge back into arr
int i = 0, j = 0, k = left;

while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {        // <= makes it STABLE
        arr[k++] = L[i++];
    } else {
        arr[k++] = R[j++];
    }
}

// Copy remaining elements (one of these will do nothing)
while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];

// Print result
for (int x = left; x <= right; x++) printf("%d ", arr[x]);
printf("\n");

free(L);
free(R);
}

// ——— MERGE SORT ———
// Strategy: Divide and Conquer – split, recurse, merge
// Time:  $O(n \log n)$  – guaranteed, even worst case
// Space:  $O(n)$  – temporary arrays during merge
// Stable: YES – equal elements maintain relative order
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;    // Avoid overflow
```

```
        // Recursively sort left and right halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("=== Merge Sort Demo ===\n\n");
    printf("Before: "); printArray(arr, n);
    printf("\nMerge trace:\n");
    mergeSort(arr, 0, n - 1);
    printf("\nAfter: "); printArray(arr, n);

    return 0;
}
```

Output:

=== Merge Sort Demo ===

Before: 38 27 43 3 9 82 10

Merge trace:

Merging [27] + [43] → 27 43

Merging [38] + [27,43] → 27 38 43

Merging [3] + [9] → 3 9

Merging [82] + [10] → 10 82

Merging [3,9] + [10,82] → 3 9 10 82

Merging [27,38,43] + [3,9,10,82] → 3 9 10 27 38 43 82

After: 3 9 10 27 38 43 82

Heap Sort — Selection Strategy

Concept

Heap Sort uses the **Heap** data structure (a complete binary tree where parent $\geq$ children, called a Max-Heap) to repeatedly select the largest element.

Key insight: The root of a Max-Heap is always the largest element. Heap Sort repeatedly extracts the max and places it at the end.

ARRAY: [4, 10, 3, 5, 1]

PHASE 1: BUILD MAX-HEAP (heapify)

Array represents a tree:

```

      4
     / \
    10  3
   / \
  5   1

```

After heapify (build max-heap):

```

     10
    / \
   5   3
  / \
 4   1

```

Array: [10, 5, 3, 4, 1]

PHASE 2: EXTRACT MAX REPEATEDLY

Step 1: Swap arr[0](max=10) with arr[4](last=1) → [1, 5, 3, 4, 10]

Heap size shrinks. Re-heapify [1, 5, 3, 4]

→ [5, 4, 3, 1] with 10 already in final place

Step 2: Swap arr[0](5) with arr[3](1) → [1, 4, 3, 5]

Re-heapify [1, 4, 3] → [4, 1, 3] with 5 in place

...continue until fully sorted: [1, 3, 4, 5, 10]

Implementation

```
#include <stdio.h>

void swap(int* a, int* b) { int t = *a; *a = *b; *b = t; }
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

// ——— HEAPIFY ———
// Ensures the subtree rooted at index 'root' satisfies Max-Heap property.
// 'n' = current heap size (shrinks as we extract max elements)
// Time: O(log n)
void heapify(int arr[], int n, int root) {
    int largest = root;    // Assume root is the largest
    int left    = 2 * root + 1; // Left child index
    int right   = 2 * root + 2; // Right child index

    // Check if left child exists and is larger than current largest
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // Check if right child exists and is larger than current largest
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is NOT the root, swap and recurse
    if (largest != root) {
        swap(&arr[root], &arr[largest]);
        heapify(arr, n, largest); // Recurse on affected subtree
    }
}
```

```

    }
}

// — HEAP SORT —————
// Strategy: Selection – build max-heap, repeatedly extract maximum
// Time:  $O(n \log n)$  – guaranteed (no worst case degradation)
// Space:  $O(1)$  – in-place sorting
// Stable: NO – heap operations can change equal elements' order
void heapSort(int arr[], int n) {
    // PHASE 1: Build Max-Heap
    // Start from last non-leaf node ( $n/2 - 1$ ) and heapify each
    printf("Phase 1: Building Max-Heap...\n");
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    printf("  Max-Heap: "); printArray(arr, n);

    // PHASE 2: Extract elements from heap one by one
    printf("\nPhase 2: Extracting max elements...\n");
    for (int i = n - 1; i > 0; i--) {
        // Move current root (max) to end
        swap(&arr[0], &arr[i]);
        printf("  Extracted max=%d → placed at index %d\n", arr[i], i);

        // Re-heapify the reduced heap (size = i, excluding sorted tail)
        heapify(arr, i, 0);
        printf("  Heap: "); printArray(arr, n);
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};

```

```
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("=== Heap Sort Demo ===\n\n");
    printf("Before: "); printArray(arr, n);
    printf("\n");
    heapSort(arr, n);
    printf("\nAfter: "); printArray(arr, n);

    return 0;
}
```

Sorting on Linked Lists

Merge Sort is uniquely suited for linked lists because it doesn't require random access.

```
// Merge Sort on a Linked List
struct Node { int data; Node* next; };

// Split list into two halves using slow/fast pointer technique
void splitList(Node* head, Node** front, Node** back) {
    Node* slow = head;
    Node* fast = head->next;

    // Fast advances 2 steps, slow advances 1 step
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    // slow is now at the midpoint
    *front = head;
    *back = slow->next;
    slow->next = NULL; // Cut the list in half
}

// Merge two sorted linked lists
Node* mergeLists(Node* a, Node* b) {
    if (!a) return b;
    if (!b) return a;

    Node* result;
    if (a->data <= b->data) {
        result = a;
        result->next = mergeLists(a->next, b);
    } else {
        result = b;
        result->next = mergeLists(a, b->next);
    }
}
```

```
    }  
    return result;  
}  
  
// Merge Sort for Linked List  
void mergeSortLL(Node** headRef) {  
    Node* head = *headRef;  
    if (!head || !head->next) return; // Base case: 0 or 1 node  
  
    Node *front, *back;  
    splitList(head, &front, &back);  
  
    mergeSortLL(&front);  
    mergeSortLL(&back);  
  
    *headRef = mergeLists(front, back);  
}
```



Section 9.6 — Exercises

Exercise 1 (Beginner): Trace Bubble Sort on [5, 1, 4, 2, 8] — draw each pass by hand. How many total swaps?

Exercise 2 (Intermediate): Implement Quick Sort with the **median-of-three** pivot selection (choose median of first, middle, and last element). Why does this help?

Exercise 3 (Advanced): Merge Sort is the best sort for linked lists. Implement `mergeSortLL()` for the singly linked list you built in section 9.2.



Sections 9.5–9.6 Quick Reference

CHOOSING THE RIGHT ALGORITHM	
Small array ($n < 20$)?	→ Bubble Sort (simple, fast enough)
Mostly sorted?	→ Bubble Sort ($O(n)$ with early exit)
General use, average perf?	→ Quick Sort (fast, cache-friendly)
Need guaranteed $O(n \log n)$?	→ Merge Sort or Heap Sort
Need stable sort?	→ Merge Sort
No extra memory allowed?	→ Heap Sort ($O(1)$ space)
Sorting a linked list?	→ Merge Sort (no random access needed)

10.0 Multi-Threading and Concurrency in C



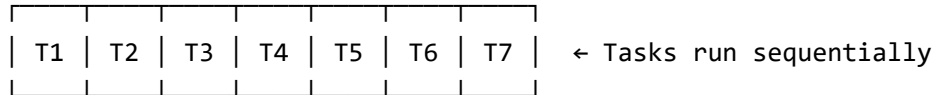
Learning Objectives

- Understand what a thread is and how it differs from a process
- Use POSIX pthreads to create, manage, and synchronize threads
- Implement mutexes, semaphores, and condition variables
- Identify and avoid race conditions and deadlocks

Introduction: Why Concurrency?

Modern computers have multiple CPU cores — yet a single-threaded program uses only ONE of them. Multi-threading lets your program run **multiple tasks simultaneously**, fully utilizing the hardware.

SINGLE-THREADED PROGRAM (1 core used):



Time: 7 units

MULTI-THREADED PROGRAM (4 cores):

Core 1:

T1	T5
----	----

Core 2:

T2	T6
----	----

Core 3:

T3	T7
----	----

Core 4:

T4

Time: ~2 units ← 3.5× faster!

Process vs Thread

PROCESS:

	Its own memory space	
	Its own file handles	
	Heavy to create	
	Communication expensive	
	(inter-process: pipes,	
	sockets, shared mem)	

THREAD:

	Shares parent's memory	
	Shares file handles	
	Lightweight to create	
	Communication cheap	
	(shared variables)	
	DANGER: race conditions!	

Threads within the same process SHARE memory – fast but dangerous!

You need SYNCHRONIZATION to prevent data corruption.

The Danger: Race Conditions

TWO THREADS updating a shared counter (counter starts at 0):

Thread 1:	Thread 2:
READ counter → 0	READ counter → 0 ← reads same value!
ADD 1 → 1	ADD 1 → 1
WRITE counter ← 1	WRITE counter ← 1 ← OVERWRITES Thread 1!

Expected result: counter = 2

Actual result: counter = 1 ← DATA CORRUPTION!

This is called a RACE CONDITION – threads racing to access shared data.

Prevention: SYNCHRONIZATION (mutex, semaphore, condition variable)

10.1 Basics of pthread Library

What is pthread?

POSIX Threads (pthreads) is the standard C library for multi-threading on Unix/Linux systems. It provides a full API for creating threads, synchronizing them, and managing their lifecycle.

PTHREAD LIBRARY OVERVIEW:

Header: `#include <pthread.h>`

Compile: `gcc program.c -lpthread -o program`

↑

Link the pthread library!

Core categories:

THREAD MANAGEMENT

`pthread_create()` → create a new thread

`pthread_join()` → wait for thread to finish

`pthread_exit()` → terminate calling thread

`pthread_self()` → get current thread's ID

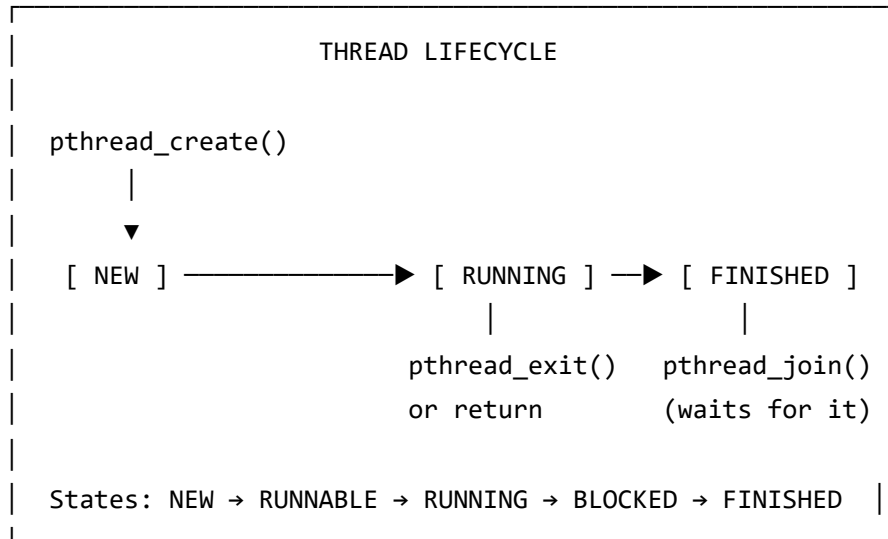
SYNCHRONIZATION

`pthread_mutex_*` → mutual exclusion locks

`sem_*` → semaphores (POSIX semaphores)

`pthread_cond_*` → condition variables

The Thread Lifecycle



10.2 Thread Creation and Management

Creating Threads

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>    // POSIX threads header
#include <unistd.h>     // sleep(), usleep()

// ——— THREAD FUNCTION SIGNATURE ———
// Every thread function MUST have this exact signature:
// void* functionName(void* arg)
// Returns: void* (can carry a result back, or NULL)
// Receives: void* arg (can be cast to any type you need)

void* helloThread(void* arg) {
    int id = *((int*)arg);    // Cast void* to int*
    printf(" Thread %d: Hello from thread!\n", id);
    sleep(1);
    printf(" Thread %d: Done.\n", id);
    return NULL;
}

// Thread function with a return value
void* computeSquare(void* arg) {
    int n = *((int*)arg);
    int* result = (int*)malloc(sizeof(int)); // Allocate result on heap!
    *result = n * n;
    printf(" Thread: %d squared = %d\n", n, *result);
}
```

```
    return (void*)result;    // Return pointer to result
}

int main() {
    printf("=== Thread Creation Demo ===\n\n");

    // — BASIC THREAD CREATION —————
    pthread_t thread1, thread2;    // Thread handles (IDs)
    int id1 = 1, id2 = 2;

    printf("Creating 2 threads...\n");

    // pthread_create(thread_handle, attributes, function, argument)
    //   - attributes: NULL means default settings
    //   - argument: cast your data pointer to void*
    if (pthread_create(&thread1, NULL, helloThread, &id1) != 0) {
        perror("Failed to create thread 1");
        return 1;
    }
    if (pthread_create(&thread2, NULL, helloThread, &id2) != 0) {
        perror("Failed to create thread 2");
        return 1;
    }

    printf("Main thread: Both threads created. Waiting...\n");

    // pthread_join: Block until thread finishes.
    // Without join, main might exit before threads complete!
    pthread_join(thread1, NULL);    // NULL = don't need return value
    pthread_join(thread2, NULL);

    printf("Main thread: Both threads finished.\n\n");
}
```

```
// — THREAD WITH RETURN VALUE —————  
printf("--- Thread Return Value Demo ---\n");  
pthread_t compThread;  
int num = 7;  
void* retval;    // Will hold thread's return value  
  
pthread_create(&compThread, NULL, computeSquare, &num);  
pthread_join(compThread, &retval);    // Capture return value  
  
int* result = (int*)retval;  
printf("Main: received result = %d\n", *result);  
free(result);    // Don't forget to free heap-allocated return value!  
  
return 0;  
}
```

Creating Multiple Threads with an Array

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5

// Structure to pass multiple arguments to a thread
struct ThreadArgs {
    int threadId;
    int workload;    // e.g., number of iterations to compute
};

void* workerThread(void* arg) {
    ThreadArgs* args = (ThreadArgs*)arg;

    long long sum = 0;
    for (int i = 1; i <= args->workload; i++)
        sum += i;

    printf("  Thread %d: sum(1..%d) = %lld\n",
           args->threadId, args->workload, sum);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    ThreadArgs args[NUM_THREADS];

    printf("=== Thread Pool Demo (%d threads) ===\n\n", NUM_THREADS);
```

```
// Create all threads
for (int i = 0; i < NUM_THREADS; i++) {
    args[i].threadId = i + 1;
    args[i].workload = (i + 1) * 1000;    // Each thread has different work

    if (pthread_create(&threads[i], NULL, workerThread, &args[i]) != 0) {
        printf("Error creating thread %d\n", i);
        return 1;
    }
}

printf("All %d threads created. Waiting for completion...\n\n", NUM_THREADS);

// Join all threads
for (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(threads[i], NULL);
}

printf("\nAll threads completed.\n");
return 0;
}
```

Thread Attributes

```
#include <stdio.h>
#include <pthread.h>

int main() {
    pthread_t thread;
    pthread_attr_t attr;    // Thread attribute object

    // Initialize attributes
    pthread_attr_init(&attr);

    // Set thread to DETACHED state (no need to join, auto-cleans up)
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);

    // Set stack size (default is typically 8MB)
    pthread_attr_setstacksize(&attr, 1024 * 1024); // 1MB stack

    // Create thread with custom attributes
    pthread_create(&thread, &attr, someFunction, NULL);

    // Destroy attributes object (not the thread!)
    pthread_attr_destroy(&attr);

    // NOTE: For detached threads, do NOT call pthread_join!
    // They clean up automatically when they finish.

    sleep(2); // Give detached thread time to finish
    return 0;
}
```

10.3 Synchronization Mechanisms in C

The Problem: Shared Data

Without synchronization:

```
global int counter = 0;
```

Thread A:	Thread B:	
LOAD counter (=0)	LOAD counter (=0)	← Both read 0!
ADD 1	ADD 1	
STORE 1	STORE 1	← Both write 1!

Expected: counter = 2

Actual: counter = 1 ← RACE CONDITION

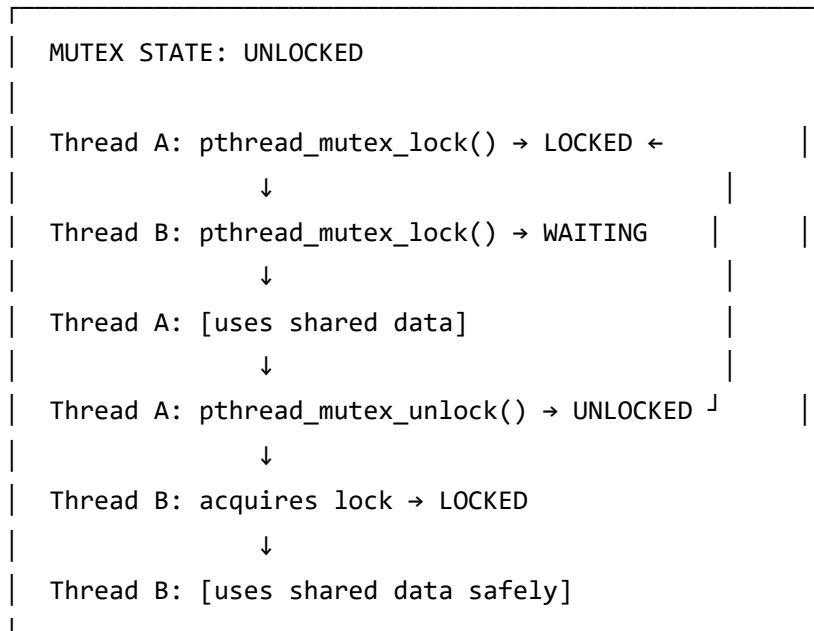
The fix: only ONE thread can touch counter at a time.

This is called MUTUAL EXCLUSION (mutex).

Mutexes

A **mutex** (MUTual EXclusion lock) is a lock that only one thread can hold at a time. It protects a **critical section** — code that accesses shared data.

MUTEX VISUALIZATION:



Mutex Implementation

```
#include <stdio.h>
#include <pthread.h>

// — SHARED DATA —————
int counter = 0;           // Shared variable (DANGER without mutex)
pthread_mutex_t counterMutex; // The mutex protecting 'counter'

// — THREAD WITHOUT MUTEX (shows the problem) —————
void* unsafeIncrement(void* arg) {
    for (int i = 0; i < 100000; i++) {
        counter++;    // NOT SAFE – race condition!
    }
    return NULL;
}

// — THREAD WITH MUTEX (correct version) —————
void* safeIncrement(void* arg) {
    for (int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&counterMutex);    // Acquire lock
        // — CRITICAL SECTION —————
        counter++;                            // Now safe!
        // — END CRITICAL SECTION —————
        pthread_mutex_unlock(&counterMutex); // Release lock
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
```

```
// — UNSAFE DEMO —————
printf("=== Mutex Demo ===\n\n");
counter = 0;
pthread_create(&t1, NULL, unsafeIncrement, NULL);
pthread_create(&t2, NULL, unsafeIncrement, NULL);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
printf("Without mutex: counter = %d (expected 200000, got %d)\n",
      counter, counter);
// Will likely print something less than 200000!

// — SAFE DEMO —————
counter = 0;
pthread_mutex_init(&counterMutex, NULL); // Initialize mutex

pthread_create(&t1, NULL, safeIncrement, NULL);
pthread_create(&t2, NULL, safeIncrement, NULL);
pthread_join(t1, NULL);
pthread_join(t2, NULL);

printf("With mutex: counter = %d (expected 200000)\n", counter);

pthread_mutex_destroy(&counterMutex); // Always clean up!
return 0;
}
```

Trylock and Timeouts

```
// pthread_mutex_trylock: Non-blocking attempt to acquire lock
// Returns 0 if acquired, EBUSY if already locked
if (pthread_mutex_trylock(&myMutex) == 0) {
    // Got the lock!
    doWork();
    pthread_mutex_unlock(&myMutex);
} else {
    // Mutex was busy, do something else
    printf("Lock is busy, skipping...\n");
}
```

Deadlock — The Mutex Danger

DEADLOCK: Two threads each hold a lock the other needs.

Thread A:	Thread B:
lock(mutex1) ✓	lock(mutex2) ✓
... (waiting)...	... (waiting)...
lock(mutex2) ← BLOCKED	lock(mutex1) ← BLOCKED

Both threads wait FOREVER. The program is stuck!

Prevention rules:

- ✓ Always acquire locks in the SAME ORDER across all threads
- ✓ Use pthread_mutex_trylock() with timeout to detect deadlock
- ✓ Keep critical sections as SHORT as possible
- ✓ Never call functions that acquire locks while holding a lock

Semaphores

A **semaphore** is a generalized synchronization tool. Unlike a mutex (which is binary: locked/unlocked), a semaphore holds an **integer count**.

SEMAPHORE ANALOGY: Parking lot with N spaces.

`sem_value = N` = number of available spaces.

Car enters (`sem_wait`): `value--` (take a space)
If value becomes negative → WAIT (lot is full)

Car leaves (`sem_post`): `value++` (free a space)
If threads waiting → wake one up

Binary semaphore (`N=1`): behaves like a mutex.

Counting semaphore (`N>1`): allows N concurrent accesses.

SEMAPHORE OPERATIONS

<code>sem_init(&sem, 0, N)</code>	→ Initialize with count N
<code>sem_wait(&sem)</code>	→ Decrement; block if count=0
<code>sem_post(&sem)</code>	→ Increment; wake a blocked thread
<code>sem_trywait(&sem)</code>	→ Non-blocking <code>sem_wait</code>
<code>sem_getvalue(&sem, &v)</code>	→ Read current count
<code>sem_destroy(&sem)</code>	→ Clean up

Semaphore Implementation

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

// ——— PRODUCER-CONSUMER PROBLEM ———
// Classic problem: one thread PRODUCES data, another CONSUMES it.
// Semaphores coordinate: consumer waits until producer has data.

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int bufferCount = 0;

sem_t emptySlots;    // Counts how many empty slots (starts at BUFFER_SIZE)
sem_t fullSlots;      // Counts how many filled slots (starts at 0)
pthread_mutex_t bufMutex;

void* producer(void* arg) {
    for (int i = 1; i <= 8; i++) {
        sem_wait(&emptySlots);    // Wait for an empty slot
        pthread_mutex_lock(&bufMutex);

        // Produce item
        buffer[bufferCount++] = i * 10;
        printf("  Producer: added %d [buffer size: %d]\n",
               i * 10, bufferCount);

        pthread_mutex_unlock(&bufMutex);
    }
}
```

```
sem_post(&fullSlots);           // Signal that a slot is now full

usleep(100000); // Simulate production time (100ms)
}
return NULL;
}

void* consumer(void* arg) {
    for (int i = 0; i < 8; i++) {
        sem_wait(&fullSlots);           // Wait until there's something to consume
        pthread_mutex_lock(&bufMutex);

        // Consume item
        int item = buffer[--bufferCount];
        printf(" Consumer: removed %d [buffer size: %d]\n",
               item, bufferCount);

        pthread_mutex_unlock(&bufMutex);
        sem_post(&emptySlots);          // Signal that a slot is now empty

        usleep(200000); // Consumer is slower (200ms)
    }
    return NULL;
}

int main() {
    pthread_t prodThread, consThread;

    // Initialize semaphores
    sem_init(&emptySlots, 0, BUFFER_SIZE); // All slots start empty
    sem_init(&fullSlots, 0, 0);           // No full slots initially
    pthread_mutex_init(&bufMutex, NULL);
```

```
printf("=== Producer-Consumer with Semaphores ===\n\n");

pthread_create(&prodThread, NULL, producer, NULL);
pthread_create(&consThread, NULL, consumer, NULL);

pthread_join(prodThread, NULL);
pthread_join(consThread, NULL);

printf("\nAll items produced and consumed.\n");

// Cleanup
sem_destroy(&emptySlots);
sem_destroy(&fullSlots);
pthread_mutex_destroy(&bufMutex);

return 0;
}
```

Output (order may vary due to timing):

```
=== Producer-Consumer with Semaphores ===
```

```
Producer: added 10 [buffer size: 1]
Producer: added 20 [buffer size: 2]
Consumer: removed 20 [buffer size: 1]
Producer: added 30 [buffer size: 2]
...
```

Condition Variables

A **condition variable** lets threads **wait for a specific condition to become true**, rather than busy-waiting (polling in a loop).

WITHOUT condition variable (BUSY WAITING – wastes CPU!):

```
while (queue_is_empty()) {  
    // do nothing, keep checking... ← burns 100% CPU!  
}  
process_item();
```

WITH condition variable (EFFICIENT):

```
pthread_mutex_lock(&mutex);  
while (queue_is_empty()) {  
    pthread_cond_wait(&cond, &mutex); // Sleep until notified  
}  
process_item();  
pthread_mutex_unlock(&mutex);
```

When producer adds item:

```
pthread_cond_signal(&cond); // Wake the waiting consumer!
```

CONDITION VARIABLE FLOW:

	Thread A (waiter):	Thread B (signaler):
	lock(mutex)	
	while(!condition)	
	cond_wait(cond, mutex)	
	↓ (ATOMICALLY:)	lock(mutex)
	releases mutex	change condition
	sleeps	cond_signal(cond)
	← woken by signal	unlock(mutex)
	re-acquires mutex	
	[condition is now true!]	
	unlock(mutex)	

Condition Variable Implementation

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// — SHARED STATE —————
int dataReady = 0;           // Condition we're waiting for
int sharedData = 0;

pthread_mutex_t dataMutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t dataCond = PTHREAD_COND_INITIALIZER;

// — CONSUMER THREAD: waits for data to be ready —————
void* consumer(void* arg) {
    pthread_mutex_lock(&dataMutex);

    // IMPORTANT: Use while(), not if() – protects against spurious wakeups
    while (!dataReady) {
        printf(" Consumer: No data yet. Waiting...\n");
        // cond_wait ATOMICALLY releases the mutex AND puts thread to sleep
        // When woken, it ATOMICALLY re-acquires the mutex
        pthread_cond_wait(&dataCond, &dataMutex);
    }

    // We have the lock and the condition is true!
    printf(" Consumer: Data received = %d\n", sharedData);
    dataReady = 0; // Reset for next round

    pthread_mutex_unlock(&dataMutex);
    return NULL;
}
```

```
// — PRODUCER THREAD: produces data and signals consumer —————  
void* producer(void* arg) {  
    sleep(2);    // Simulate time to prepare data  
  
    pthread_mutex_lock(&dataMutex);  
  
    sharedData = 42;  
    dataReady  = 1;  
    printf("  Producer: Data prepared = %d. Signaling consumer.\n", sharedData);  
  
    pthread_cond_signal(&dataMutex);    // Wake ONE waiting thread  
    // Use pthread_cond_broadcast() to wake ALL waiting threads  
  
    pthread_mutex_unlock(&dataMutex);  
    return NULL;  
}  
  
// — ADVANCED: THREAD-SAFE MESSAGE QUEUE —————  
// A production-grade pattern using condition variables  
#define QUEUE_MAX 10  
  
struct MsgQueue {  
    int items[QUEUE_MAX];  
    int head, tail, count;  
    pthread_mutex_t lock;  
    pthread_cond_t  notEmpty;  
    pthread_cond_t  notFull;  
};  
  
void mq_init(MsgQueue* q) {  
    q->head = q->tail = q->count = 0;
```

```
pthread_mutex_init(&q->lock, NULL);
pthread_cond_init(&q->notEmpty, NULL);
pthread_cond_init(&q->notFull, NULL);
}

void mq_push(MsgQueue* q, int value) {
    pthread_mutex_lock(&q->lock);
    while (q->count == QUEUE_MAX)           // Queue full → wait
        pthread_cond_wait(&q->notFull, &q->lock);

    q->items[q->tail] = value;
    q->tail = (q->tail + 1) % QUEUE_MAX;
    q->count++;

    pthread_cond_signal(&q->notEmpty);       // Wake a consumer
    pthread_mutex_unlock(&q->lock);
}

int mq_pop(MsgQueue* q) {
    pthread_mutex_lock(&q->lock);
    while (q->count == 0)                   // Queue empty → wait
        pthread_cond_wait(&q->notEmpty, &q->lock);

    int value = q->items[q->head];
    q->head = (q->head + 1) % QUEUE_MAX;
    q->count--;

    pthread_cond_signal(&q->notFull);       // Wake a producer
    pthread_mutex_unlock(&q->lock);
    return value;
}
```

```
int main() {  
    pthread_t prod, cons;  
  
    printf("=== Condition Variable Demo ===\n\n");  
  
    pthread_create(&cons, NULL, consumer, NULL);  
    pthread_create(&prod, NULL, producer, NULL);  
  
    pthread_join(prod, NULL);  
    pthread_join(cons, NULL);  
  
    printf("\nDone.\n");  
  
    pthread_mutex_destroy(&dataMutex);  
    pthread_cond_destroy(&dataCond);  
    return 0;  
}
```

Synchronization Comparison

SYNCHRONIZATION MECHANISMS COMPARISON	
Mechanism	Use Case
Mutex	Exclusive access to shared variable/object "Only 1 thread at a time in this section"
Semaphore	Limit concurrent access (N threads allowed) Signal between threads (producer-consumer) "Up to N threads can use this resource"
Condition Var	Wait for a specific state/event "Sleep until this condition becomes true" Always used WITH a mutex



Common Multi-Threading Mistakes

✗ MISTAKE 1: Forgetting -lpthread when compiling

```
gcc program.c -o program          ← linker error!  
gcc program.c -lpthread -o program ← correct
```

✗ MISTAKE 2: Passing stack variable address to thread

```
int localVar = 5;  
pthread_create(&t, NULL, fn, &localVar);  
// localVar might go out of scope before thread uses it!  
// FIX: use malloc() or a global/static variable
```

✗ MISTAKE 3: Forgetting to join detached threads

```
pthread_create(&t, ...);  
return 0; // Main exits, thread is killed mid-execution!  
// FIX: pthread_join(), or mark thread as detached
```

✗ MISTAKE 4: Unlocking a mutex you don't own

Thread A locks mutex, Thread B tries to unlock it → undefined!

✗ MISTAKE 5: Not using while() with cond_wait (spurious wakeups)

```
if (!ready) cond_wait(...) ← WRONG: spurious wakeup bypasses check  
while (!ready) cond_wait(...) ← CORRECT
```



Best Practices for Multi-Threading

- ✓ Keep critical sections as short as possible (reduce contention)
- ✓ Always lock/unlock in the same thread
- ✓ Always acquire multiple locks in the same order (prevent deadlock)
- ✓ Prefer condition variables over busy-waiting
- ✓ Use `pthread_mutex_trylock()` to detect potential deadlocks in debug
- ✓ Test with thread sanitizers: `gcc -fsanitize=thread`
- ✓ Design for lock-free where possible (atomic operations)
- ✓ Initialize all mutexes, semaphores, cond vars before use
- ✓ Destroy all synchronization objects when done



Section 10 — Exercises

Exercise 1 (Beginner): Create 5 threads that each print "Hello from thread N" (where N is their ID). Ensure all threads finish before main exits.

Exercise 2 (Intermediate): Implement a thread-safe stack using a mutex. Two threads simultaneously push and pop 1000 items each. Verify no data corruption.

Exercise 3 (Advanced): Implement the Dining Philosophers problem (5 philosophers, 5 forks). First without synchronization to observe deadlock, then with a solution using mutexes or semaphores.

11.0 Search and Sorting Algorithms — Applied

Introduction

Chapter 11 bridges theory and practice — applying the algorithms from chapters 9.5 and 9.6 to realistic problems and data structures. We go deeper on implementation nuances, apply them to linked lists, and explore advanced patterns.

11.1 Implementing Linear and Binary Search

Applied Search on Different Data Structures

Search in a 2D Array (Matrix)

```
#include <stdio.h>

#define ROWS 4
#define COLS 4

// ——— LINEAR SEARCH IN 2D ARRAY ———
// Treats the matrix as a flat sequence
void matrixLinearSearch(int matrix[ROWS][COLS], int target) {
    printf("Linear search in %dx%d matrix for %d:\n", ROWS, COLS, target);

    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (matrix[i][j] == target) {
                printf(" Found at row=%d, col=%d\n", i, j);
                return;
            }
        }
    }
    printf(" Not found.\n");
}

// ——— BINARY SEARCH IN SORTED MATRIX ———
```

```
// Special property: each row is sorted, and row[i][COLS-1] < row[i+1][0]
// This lets us treat the whole matrix as one sorted array.
// Time: O(log(ROWS * COLS))
```

```
int matrixBinarySearch(int matrix[ROWS][COLS], int target) {
    int low = 0;
    int high = ROWS * COLS - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        // Convert flat index to 2D: row = mid/COLS, col = mid%COLS
        int midVal = matrix[mid / COLS][mid % COLS];

        if (midVal == target) return mid;
        else if (midVal < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

```
int main() {
    int matrix[ROWS][COLS] = {
        { 1,  3,  5,  7},
        { 9, 11, 13, 15},
        {17, 19, 21, 23},
        {25, 27, 29, 31}
    };

    printf("Matrix:\n");
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++)
            printf("%3d ", matrix[i][j]);
        printf("\n");
    }
}
```

```
    }  
    printf("\n");  
  
    matrixLinearSearch(matrix, 19);  
  
    int idx = matrixBinarySearch(matrix, 19);  
    if (idx != -1)  
        printf("Binary search: found at row=%d, col=%d\n",  
            idx / COLS, idx % COLS);  
  
    return 0;  
}
```

Binary Search Variants

```
// — LOWER BOUND —————  
// Find the first index where arr[i] >= target  
// Useful for sorted arrays with duplicates  
int lowerBound(int arr[], int n, int target) {  
    int low = 0, high = n;    // Note: high = n (one past end)  
    while (low < high) {  
        int mid = low + (high - low) / 2;  
        if (arr[mid] < target)  
            low = mid + 1;  
        else  
            high = mid;  
    }  
    return low;    // First index where arr[i] >= target  
}  
  
// — UPPER BOUND —————  
// Find the first index where arr[i] > target  
int upperBound(int arr[], int n, int target) {  
    int low = 0, high = n;  
    while (low < high) {  
        int mid = low + (high - low) / 2;  
        if (arr[mid] <= target)  
            low = mid + 1;  
        else  
            high = mid;  
    }  
    return low;    // First index where arr[i] > target  
}
```

```
// Count occurrences using lower and upper bound
int countOccurrences(int arr[], int n, int target) {
    return upperBound(arr, n, target) - lowerBound(arr, n, target);
}

// Usage:
// int arr[] = {1, 2, 3, 3, 3, 4, 5};
// countOccurrences(arr, 7, 3) → returns 3 (indices 2,3,4)
```

Search on a Sorted Linked List

```
struct Node { int data; Node* next; };

// ——— LINEAR SEARCH ON LINKED LIST ———
// Binary search is NOT possible on a linked list (no random access)
Node* searchLL(Node* head, int target) {
    Node* curr = head;
    int pos = 0;

    while (curr != NULL) {
        if (curr->data == target) {
            printf("Found %d at position %d\n", target, pos);
            return curr;
        }
        // OPTIMIZATION: if sorted and curr > target, stop early
        if (curr->data > target) {
            printf("%d not in list (stopped early at %d)\n", target, curr->data);
            return NULL;
        }
        curr = curr->next;
        pos++;
    }
    printf("%d not found.\n", target);
    return NULL;
}
```

11.2 Sorting Algorithms: Bubble, Quick, Merge Sort

Sorting with Custom Comparators

Real programs sort objects by multiple fields. Here we sort structs.

```
#include <stdio.h>
#include <string.h>
```

```
struct Student {
    char name[30];
    int grade;
    float gpa;
};
```

```
// — SORT STUDENTS BY GRADE (BUBBLE SORT) —————
```

```
void sortByGrade(Student students[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++) {
            if (students[j].grade > students[j+1].grade) {
                // Swap entire Student structs
                Student temp = students[j];
                students[j] = students[j+1];
                students[j+1] = temp;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}
```

```
// — SORT STUDENTS BY NAME (MERGE SORT) —————
```

```
void mergeStudents(Student arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
```

```

Student* L = (Student*)malloc(n1 * sizeof(Student));
Student* R = (Student*)malloc(n2 * sizeof(Student));

for (int i = 0; i < n1; i++) L[i] = arr[left + i];
for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
    // strcmp: negative if L < R alphabetically
    if (strcmp(L[i].name, R[j].name) <= 0)
        arr[k++] = L[i++];
    else
        arr[k++] = R[j++];
}
while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];

free(L); free(R);
}

void mergeSortByName(Student arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSortByName(arr, left, mid);
        mergeSortByName(arr, mid + 1, right);
        mergeStudents(arr, left, mid, right);
    }
}

void printStudents(Student s[], int n) {
    printf("%-12s %5s %5s\n", "Name", "Grade", "GPA");
    printf("%-12s %5s %5s\n", "—————", "———", "———");
}

```

```
    for (int i = 0; i < n; i++)
        printf("%-12s %5d %5.2f\n", s[i].name, s[i].grade, s[i].gpa);
    printf("\n");
}

int main() {
    Student students[] = {
        {"Charlie", 85, 3.5},
        {"Alice", 92, 3.9},
        {"Bob", 78, 3.2},
        {"Diana", 95, 4.0},
        {"Eve", 88, 3.7}
    };
    int n = 5;

    printf("=== Sorting Structs Demo ===\n\n");

    printf("Original:\n");
    printStudents(students, n);

    sortByGrade(students, n);
    printf("Sorted by grade (Bubble Sort):\n");
    printStudents(students, n);

    mergeSortByName(students, 0, n - 1);
    printf("Sorted by name (Merge Sort):\n");
    printStudents(students, n);

    return 0;
}
```

Output:

=== Sorting Structs Demo ===

Original:

Name	Grade	GPA
Charlie	85	3.50
Alice	92	3.90
Bob	78	3.20
Diana	95	4.00
Eve	88	3.70

Sorted by grade (Bubble Sort):

Name	Grade	GPA
Bob	78	3.20
Charlie	85	3.50
Eve	88	3.70
Alice	92	3.90
Diana	95	4.00

Sorted by name (Merge Sort):

Name	Grade	GPA
Alice	92	3.90
Bob	78	3.20
Charlie	85	3.50
Diana	95	4.00
Eve	88	3.70

Advanced Quick Sort: Three-Way Partitioning (Dutch National Flag)

Standard Quick Sort is slow when many equal elements exist ($O(n^2)$). Three-way partitioning handles duplicates in $O(n \log n)$:

```
// — THREE-WAY QUICKSORT —————  
// Partitions into: [< pivot] [== pivot] [> pivot]  
// Excellent for arrays with many duplicate keys  
void quickSort3Way(int arr[], int low, int high) {  
    if (low >= high) return;  
  
    int pivot = arr[low];  
    int lt = low;    // arr[low..lt-1] < pivot  
    int gt = high;   // arr[gt+1..high] > pivot  
    int i = low + 1; // arr[lt..i-1] == pivot  
  
    while (i <= gt) {  
        if (arr[i] < pivot) {  
            swap(&arr[lt], &arr[i]);  
            lt++; i++;  
        } else if (arr[i] > pivot) {  
            swap(&arr[i], &arr[gt]);  
            gt--;  
        } else {  
            i++;  
        }  
    }  
    // arr[low..lt-1] < pivot, arr[lt..gt] == pivot, arr[gt+1..high] > pivot  
    quickSort3Way(arr, low, lt - 1);  
    quickSort3Way(arr, gt + 1, high);  
}  
  
// Usage:  
// int arr[] = {4, 3, 4, 4, 1, 4, 2, 4};  
// quickSort3Way(arr, 0, 7);
```

```
// Result: [1, 2, 3, 4, 4, 4, 4, 4]
```

```
// All 4s are placed in one step – no recursion on them!
```

Performance Benchmarking

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

void bubbleSortBench(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int sw = 0;
        for (int j = 0; j < n-1-i; j++) {
            if (arr[j] > arr[j+1]) {
                int t = arr[j]; arr[j] = arr[j+1]; arr[j+1] = t;
                sw = 1;
            }
        }
        if (!sw) break;
    }
}

int compareFn(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

void benchmark(const char* name, void (*sortFn)(int[], int),
               int* data, int n) {
    int* copy = (int*)malloc(n * sizeof(int));
    memcpy(copy, data, n * sizeof(int));

    clock_t start = clock();
```

```
    sortFn(copy, n);
    clock_t end = clock();

    double elapsed = (double)(end - start) / CLOCKS_PER_SEC * 1000;
    printf(" %-15s: %.3f ms\n", name, elapsed);
    free(copy);
}

int main() {
    int N = 20000;
    int* data = (int*)malloc(N * sizeof(int));
    srand(42);
    for (int i = 0; i < N; i++) data[i] = rand() % 100000;

    printf("=== Sort Benchmark (%d elements) ===\n\n", N);
    benchmark("Bubble Sort", bubbleSortBench, data, N);
    // Add quickSort, mergeSort benchmarks similarly
    // stdlib qsort for comparison:
    int* copy = (int*)malloc(N * sizeof(int));
    memcpy(copy, data, N * sizeof(int));
    clock_t s = clock();
    qsort(copy, N, sizeof(int), compareFn);
    clock_t e = clock();
    printf(" %-15s: %.3f ms\n", "stdlib qsort",
           (double)(e-s)/CLOCKS_PER_SEC*1000);
    free(copy); free(data);
    return 0;
}
```



Chapter 11 — Final Exercises

Exercise 1 (Beginner): Sort a 2D matrix row by row using Bubble Sort, then search for a specific value using Binary Search on each row.

Exercise 2 (Intermediate): Implement a function that checks if an array of strings is sorted in alphabetical order using Linear Search logic (compare adjacent pairs).

Exercise 3 (Advanced): Implement an **external sort** simulation: you have 1,000,000 integers but only 100,000 fit in memory. Design a Merge Sort–based solution that sorts in chunks and merges.



Final Master Quick Reference

COMPLETE ALGORITHM REFERENCE CARD				
Algorithm	Best	Average	Worst	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Jump Search	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
pthread ops	$O(1)$	$O(1)$	$O(1)$	$O(n)$

SYNCHRONIZATION QUICK PICK:

- 1 thread at a time? → Mutex
- N threads at a time? → Semaphore (count = N)
- Wait for an event? → Condition Variable
- Signal one vs all? → cond_signal vs cond_broadcast



Additional Resources

Books

- **"The Art of Computer Programming" by Donald Knuth (Vol 3: Sorting & Searching)** — The definitive reference. Exhaustive and authoritative.
- **"Introduction to Algorithms" (CLRS) by Cormen et al.** — The standard university textbook for algorithms.
- **"Programming with POSIX Threads" by David Butenhof** — The gold standard book for pthreads.
- **"Parallel Programming in C with MPI and OpenMP" by Quinn** — Next step after pthreads mastery.

Tools

- **Valgrind** (`valgrind --tool=helgrind ./program`) — Detects race conditions and thread errors
- **GCC Thread Sanitizer** (`gcc -fsanitize=thread`) — Fast, built-in race condition detection
- **VisuAlgo.net** — Animated sort/search visualizations
- **Sorting.at** — Beautiful visual comparison of sorting algorithms in real time

Practice Platforms

- **LeetCode** → Search "sorting" and "binary search" tags for 200+ problems
- **HackerRank** → "Algorithms" section covers all topics in this guide
- **Codeforces** → Competitive programming problems requiring fast search/sort



Pro Tip for Multi-Threading: The golden rule is: *"If two threads can touch it, protect it."* When in doubt, add a mutex. Profile later if performance is an issue — premature optimization is the root of all threading bugs.

End of Part 2: Chapters 9.5, 9.6, 10.0, 11.0
Data Structures and Operations Using C++ in C Style
Guide Version 1.0 | Complete Mastery Edition